# SIMATS ENGINEERING

## CO5 ASSESSMENT TOOL 1

### APPLICATION-BASED QUESTIONS (High)

| | | |
|---|---|---|
| **Course: ETHICAL HACKING** | **Code: ITA1416** | **CO:** CO5 |
| **Duration:** | **Total Marks: 100** | **Reg No / Name :**<br>G.RAJU (192411065) |

---

**COURSE OUTCOME COVERED**

***CO5:*** *Analyze vulnerabilities in Windows and Linux systems and evaluate security weaknesses in messaging, web, and mobile applications using appropriate exploitation techniques and hacking tools. (BT5)*

---

*Instructions: Answer all questions. Each question carries 20 marks.*

**1. A mid-sized financial organization operates several legacy Windows servers where multiple RPC services remain exposed due to delayed patching cycles caused by operational constraints. Recently, abnormal traffic patterns and unauthorized connection attempts have been observed within the internal network. As a security analyst, evaluate how attackers could exploit these RPC vulnerabilities to gain unauthorized access and escalate privileges. Assess the level of risk associated with delaying patches in such environments and justify whether immediate patch deployment or controlled maintenance scheduling is more appropriate. Recommend suitable mitigation strategies to reduce exposure without affecting business continuity.**

**Introduction**

Remote Procedure Call (RPC) is a Windows mechanism that allows processes and services to communicate over a network. If RPC services are exposed and remain unpatched, attackers can exploit known vulnerabilities to compromise systems. In a financial organization, such vulnerabilities pose a serious security risk because servers may contain sensitive financial and customer information.

**How Attackers Could Exploit RPC Vulnerabilities**

1. **Network Reconnaissance:** Attackers first identify Windows servers and exposed RPCrelated services using network scanning and service enumeration.
2. **Vulnerability Identification:** They determine the operating system and RPC service versions and compare them against known vulnerabilities.
3. **Remote Exploitation:** If a vulnerable RPC service is exposed, an attacker may send specially crafted requests that trigger unauthorized behavior or remote code execution.
4. **Initial Access:** Successful exploitation can provide the attacker with access to the affected server.
5. **Privilege Escalation:** If the vulnerable service operates with administrative or SYSTEM privileges, exploitation may result in elevated privileges.
6. **Credential Theft:** Attackers may attempt to obtain credentials or authentication tokens from the compromised system.
7. **Lateral Movement:** The compromised server can become a starting point for accessing other internal systems.
8. **Data Theft or Disruption:** Attackers may steal financial information, install malware, deploy ransomware, or disrupt services.

**Risk of Delaying Patches**

The risk is High to Critical because:

- Known vulnerabilities can have publicly available exploits.
- Legacy systems may have weaker security configurations.
- Attackers can automate vulnerability scanning.

- A single compromised server can facilitate lateral movement.
- Financial organizations are high-value targets.
- Delayed patching increases the organization's attack window.
- Compromise may result in financial loss, data breaches, regulatory penalties, and reputational damage.

**Immediate Patching vs Controlled Maintenance**

Immediate patching is preferred for critical RPC vulnerabilities, especially on internet-facing or highly exposed systems.

However, financial organizations may have applications dependent on legacy servers.
Therefore, the organization can:

- Test patches in a staging environment.
- Schedule controlled maintenance.
- Prepare rollback procedures.
- Patch the most critical systems first.
- Use temporary compensating controls until patching is complete.

Delaying patch deployment without compensating controls is not recommended.

**Mitigation Strategies**

- Apply the latest security patches.
- Disable unnecessary RPC services.
- Restrict RPC access using firewalls.
- Allow connections only from trusted systems.
- Implement network segmentation.
- Use host-based firewalls.
- Monitor RPC-related traffic and Windows event logs.
- Deploy endpoint detection and response.

- Conduct regular vulnerability assessments.
- Maintain offline and tested backups.
- Establish a formal patch-management policy.
- Maintain an inventory of legacy systems.

**Conclusion**

Unpatched RPC services represent a significant attack surface. Immediate patching should be the primary approach, while controlled maintenance and network isolation can be used when operational constraints exist. Combining patch management, segmentation, monitoring, and access control can reduce the risk without significantly affecting business continuity.

**2. A web application used for online transactions suffers from multiple vulnerabilities, including cross-site scripting (XSS), cross-site request forgery (CSRF), and weak session management practices. Users report unauthorized transactions and session hijacking incidents. Evaluate how attackers could combine these vulnerabilities to execute complex attacks affecting user accounts. Assess the risks associated with poor session handling and lack of input validation. Justify the need for a layered security approach and recommend comprehensive mitigation strategies to secure the application.**

**Introduction**

Web applications handling online transactions must protect authentication sessions and user inputs. Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and weak session management can individually create security problems, but when combined, they can enable complex attacks such as account takeover and unauthorized transactions.

**How Attackers Can Combine the Vulnerabilities**

1. **XSS Injection:** The attacker inserts malicious JavaScript through an insufficiently validated input field.
2. **Script Execution:** When another user views the affected page, the malicious script executes in the user's browser.

3. **Session Abuse:** If session protection is weak, the attacker may abuse authentication information or perform actions within the victim's session.
4. **CSRF Attack:** The attacker can cause the victim's browser to send unauthorized requests to the application.
5. **Unauthorized Transactions:** Because the victim is already authenticated, the server may incorrectly treat the malicious request as legitimate.
6. **Account Takeover:** Stolen or abused authentication information can allow continued unauthorized access.
7. **Data Theft:** Personal and financial information may be exposed.
8. **Persistence:** Attackers may modify account settings or establish mechanisms that allow continued access.

**Risks of Poor Session Management** Weak

session management may involve:

- Predictable session identifiers.
- Sessions that never expire.
- Failure to invalidate sessions after logout.
- Session IDs remaining unchanged after authentication.
- Missing Secure and HttpOnly cookie attributes.
- Excessively long session lifetimes.

These weaknesses increase the possibility of session hijacking and account takeover.

**Risks of Poor Input Validation**

Poor input validation allows attackers to insert malicious content into application fields. This can result in:

- XSS attacks.
- Data manipulation.
- Account compromise.

- Unauthorized actions.
- Information disclosure.
- Damage to application integrity.

**Need for Layered Security**

A single security mechanism cannot protect against every attack. For example, CSRF protection does not prevent XSS, while input validation alone does not provide secure session management.

Therefore, defense in depth should be implemented.

**Mitigation Strategies For**

**XSS:**

- Validate input.
- Sanitize untrusted content.
- Apply context-aware output encoding.
- Implement Content Security Policy.
- Avoid unsafe JavaScript execution.

**For CSRF:**

- Use anti-CSRF tokens.
- Configure SameSite cookies.
- Validate request origin where appropriate.
- Require additional verification for sensitive transactions.

**For Session Security:**

- Use random session identifiers.
- Regenerate sessions after authentication.
- Implement session timeout.
- Invalidate sessions after logout.

- Use Secure, HttpOnly, and appropriate SameSite cookie attributes.
- Implement multi-factor authentication.

**Additional Measures:**

- Perform penetration testing.
- Use secure coding standards.
- Monitor suspicious transactions.
- Implement transaction confirmation for high-risk operations.
- Maintain detailed security logs.

**Conclusion**

XSS, CSRF, and weak session management can combine to cause serious account compromise and unauthorized financial transactions. A layered security architecture involving secure coding, input validation, session protection, CSRF defenses, authentication, monitoring, and transaction verification is essential.

**3. An enterprise uses an outdated IIS server to host multiple internal and external applications, and recent vulnerability scans indicate several unpatched critical flaws. Despite warnings, the organization delays updates due to fear of downtime and service disruption. Assess how attackers could exploit these vulnerabilities to gain unauthorized access or execute remote code. Evaluate the risks associated with delaying security updates in such environments and justify whether immediate patching or temporary isolation is more appropriate. Recommend strategies to ensure both security and service availability.**

**Introduction**

Internet Information Services (IIS) is a widely used Microsoft web server platform. An outdated IIS installation containing critical vulnerabilities can provide attackers with opportunities to compromise web applications, execute malicious code, and gain access to underlying Windows systems.

**How Attackers Could Exploit the Vulnerabilities**

1. **Information Gathering:** Attackers identify the IIS server and determine its version and exposed applications.
2. **Vulnerability Scanning:** They compare the server configuration against known vulnerabilities.
3. **Exploitation:** Attackers attempt to exploit vulnerable IIS components or applications.
4. **Remote Code Execution:** A critical vulnerability may allow malicious code to execute remotely.
5. **Privilege Escalation:** Attackers may attempt to move from application-level access to higher system privileges.
6. **Credential Theft:** Compromised systems can be used to obtain credentials.
7. **Lateral Movement:** Attackers can use the server to reach other internal systems.
8. **Data Exfiltration:** Sensitive customer or organizational data may be stolen.
9. **Malware/Ransomware:** Attackers may install malicious software.
10. **Service Disruption:** Critical applications may be disabled or disrupted. **Risks of Delaying Security Updates** The risk is Critical because:

- Vulnerabilities are already known.
- Exploits may be publicly available.
- Internet-facing servers are continuously scanned.
- Automated attacks can compromise vulnerable systems quickly.
- A compromised web server can expose backend databases.
- Business downtime caused by an attack may be greater than planned maintenance downtime.

**Immediate Patching vs Temporary Isolation**

Immediate patching should be the preferred approach for critical vulnerabilities.

If patching cannot immediately be performed because of business dependencies, temporary isolation should be used, such as:

- Restricting network access.
- Placing the server behind a WAF.
- Limiting administrative access.
- Disabling vulnerable components where possible.
- Increasing monitoring.

Temporary isolation should only be considered a short-term compensating control, not a permanent replacement for patching.

**Strategies for Security and Availability** •

Test patches in a staging environment.

- Use scheduled maintenance windows.
- Create system backups.
- Maintain rollback procedures.
- Use load balancing where possible.
- Patch redundant servers sequentially.
- Deploy a WAF.
- Restrict unnecessary ports and services.
- Monitor IIS logs.
- Use endpoint security.
- Perform continuous vulnerability scanning.
- Maintain disaster recovery procedures.

**Conclusion**

Delaying critical IIS updates exposes the organization to potentially severe attacks. Immediate patching is the best solution, while temporary isolation, WAF protection, segmentation, and monitoring can reduce risk until patching is safely completed.

**4. A mobile application used by customers stores sensitive information such as login credentials and personal data in plaintext within the device storage. Additionally, the application communicates with backend servers over unsecured channels. Evaluate how attackers could exploit these weaknesses through device compromise or network interception. Assess the potential impact on user privacy and organizational reputation. Justify the need for encryption and recommend secure storage and communication techniques.**

**Introduction**

Mobile applications frequently process sensitive information such as credentials, personal data, authentication tokens, and customer information. Storing such data in plaintext and communicating with backend servers over unsecured channels can expose users to serious privacy and security threats.

**Exploitation Through Device Compromise**

If an attacker gains access to a compromised or rooted device, plaintext application data may be accessible.

Attackers could:

1. Access application storage.
2. Search for plaintext credentials.
3. Obtain authentication tokens.
4. Retrieve personal information.
5. Use stolen credentials to access user accounts.
6. Extract sensitive information from application databases or files.

**Exploitation Through Network Interception**

Unsecured communication can allow attackers positioned on an untrusted network to observe or manipulate network traffic.

Potential consequences include:

- Credential interception.
- Session/token theft.
- Personal-data exposure.
- Modification of transmitted information.
- Man-in-the-middle attacks.
- Account compromise.

**Impact on Users** The weaknesses can result in:

- Privacy violations.
- Identity theft.
- Account takeover.
- Financial loss.
- Credential reuse attacks.
- Exposure of personal information.

**Organizational Impact** The organization may experience:

- Loss of customer trust.
- Reputational damage.
- Regulatory consequences.
- Financial losses.
- Incident-response costs.

- Legal consequences.
- Loss of business.

**Secure Storage Techniques**

Sensitive information should not be stored in plaintext.

Recommended techniques include:

- Android Keystore.
- iOS Keychain.
- Hardware-backed key storage where available.
- Encryption of sensitive local databases/files.
- Secure authentication tokens.
- Minimal local data storage.
- Secure deletion of unnecessary sensitive information.

Passwords should never be stored as plaintext.

**Secure Communication Techniques** •

Use HTTPS for all communication.

- Use modern TLS configurations.
- Properly validate server certificates.
- Protect authentication tokens.
- Use secure API authentication.
- Consider certificate pinning where appropriate.
- Reject insecure HTTP connections.

**Conclusion**

Mobile applications must protect sensitive information both at rest and in transit. Strong encryption, secure platform storage, TLS communication, secure authentication, and minimal data storage significantly reduce the risk of device compromise and network interception.

**5. A system administrator disables the Windows firewall temporarily to troubleshoot connectivity issues but forgets to re-enable it. Within a short period, multiple unauthorized access attempts are detected. Evaluate how attackers could exploit this temporary exposure to gain access to critical services. Assess the risks associated with mismanagement of security controls and justify the importance of enforcing automated policies. Recommend administrative safeguards to prevent such incidents.**

**Introduction**

A firewall acts as an important security boundary by controlling incoming and outgoing network connections. Accidentally leaving the Windows firewall disabled can expose network services and increase the attack surface of the system.

**How Attackers Could Exploit the Exposure**

1. **Network Scanning:** Attackers scan the exposed system for open ports and services.
2. **Service Identification:** They identify accessible Windows services.
3. **Vulnerability Detection:** Attackers determine whether exposed services contain known weaknesses.
4. **Unauthorized Access:** They attempt authentication attacks or exploit vulnerable services.
5. **Malware Installation:** Successful access may allow malicious software to be installed.
6. **Privilege Escalation:** Attackers may attempt to obtain administrative privileges.
7. **Lateral Movement:** A compromised system can be used to attack other machines.
8. **Data Theft:** Sensitive information may be accessed or transferred.
9. **Service Disruption:** Attackers may disable applications or critical services.

**Risks of Mismanaging Security Controls**

The incident demonstrates that even correctly configured security controls can become ineffective because of administrative errors.

Major risks include:

- Increased attack surface.
- Unauthorized network access.
- Malware infection.
- Data theft.
- Privilege escalation.
- Lateral movement.
- Business disruption.
- Compliance violations.

**Importance of Automated Policies**

Security controls should not depend entirely on manual actions.

Automated policies can:

- Prevent unauthorized firewall changes.
- Automatically restore required configurations.
- Detect disabled security controls.
- Generate alerts.
- Enforce organizational security standards.
- Continuously monitor compliance.

For Windows environments, centralized management such as Group Policy can be used to enforce firewall configurations.

**Administrative Safeguards**

- Enforce firewall settings through centralized policies.
- Use least-privilege administrative accounts.
- Require approval for security-control changes.
- Log all configuration changes.

- Generate alerts when the firewall is disabled.
- Perform automated compliance checks.
- Use endpoint security solutions.
- Maintain configuration baselines.
- Set temporary exceptions with automatic expiration.
- Conduct regular security audits.
- Train administrators on secure troubleshooting procedures.
- Monitor network traffic for unexpected connections.

**Incident Response**

Once the firewall is found to be disabled:

1. Re-enable the firewall.
2. Review security and system logs.
3. Identify unauthorized connection attempts.
4. Check for suspicious processes or files.
5. Verify that critical services were not compromised.
6. Reset potentially exposed credentials if necessary.
7. Perform vulnerability scanning.
8. Document the incident.
9. Improve controls to prevent recurrence.

**Conclusion**

Disabling the firewall, even temporarily, can create a significant security exposure. Automated configuration enforcement, monitoring, least privilege, change management, and administrative safeguards are necessary to prevent human errors from becoming serious security incidents.

**Code of Conduct:**

*I* **G.RAJU (192411065)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

*G.RAJU*

***Signature of the student:***

**RUBRICS FOR CALCULATION**

| **Criteria** | **Weightage** | **Level 4 – Exemplary** | **Level 3 – Proficient** | **Level 2 – Developing** | **Level 1 – Beginning** | **Mark** |
|---|---|---|---|---|---|---|
| Understanding of Problem Context | 20 | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem | |
| Application of Concepts | 30 | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts | |
| Problem-Solving Approach | 20 | Logical, wellstructured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach | |
| Accuracy of Solution | 20 | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution | |
| Clarity | 10 | Clear, wellorganized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand | |